Assignment

Operating System Lab

Submitted By:

Name: Mahamudul Hasan Masum
ID: 2310876111
Dept: CSE
Session: 2022-23

1. Peterson's Solution

```cpp
#include <iostream>
#include <thread>

bool flag[2] = {false, false};
int turn;

void process(int process_id) {
    int other = 1 - process_id;
    flag[process_id] = true;
    turn = process_id;
    while (flag[other] && turn == process_id) {}

    // Critical Section

    flag[process_id] = false;
}

int main() {
    std::thread t1(process, 0);
    std::thread t2(process, 1);
    t1.join();
    t2.join();
    return 0;
}
```

2. Strict Alternation

```cpp
#include <iostream>
#include <thread>

int turn = 0;

void process(int process_id) {
    while (true) {
        while (turn != process_id) {}

        // Critical Section

        turn = 1 - process_id;
    }
}

int main() {
    std::thread t1(process, 0);
    std::thread t2(process, 1);
    t1.detach();
    t2.detach();
    return 0;
}
```

3. Sleep and Wakeup (Producer-Consumer Problem)

```cpp
#include <iostream>

#define N 100
int count = 0;

void sleep() {}
void wakeup(void (*func)()) {}
void produce_item() {}
void consume_item() {}
void enter_item() {}
void remove_item() {}

```

```
void producer() {
    while (true) {
        produce_item();
        if (count == N) sleep();
        enter_item();
        count = count + 1;
        if (count == 1) wakeup(reinterpret_cast<void (*)()>(1));
    }
}

void consumer() {
    while (true) {
        if (count == 0) sleep();
        remove_item();
        count = count - 1;
        if (count == N - 1) wakeup(reinterpret_cast<void (*)()>(0));
        consume_item();
    }
}
```

4. TSL Instruction

```
#include <atomic>
#include <thread>

std::atomic_flag lock = ATOMIC_FLAG_INIT;

void process() {
    while (lock.test_and_set(std::memory_order_acquire)) {}

    // Critical Section

    lock.clear(std::memory_order_release);
}

int main() {
    std::thread t1(process);
    std::thread t2(process);
    t1.join();
    t2.join();
    return 0;
}
```

5. XCHG Instruction

```
#include <atomic>
#include <thread>

std::atomic<bool> lock(false);

void process() {
    bool key = true;
    while (key) {
        key = lock.exchange(true, std::memory_order_acquire);
    }

    // Critical Section

    lock.store(false, std::memory_order_release);
}

int main() {
    std::thread t1(process);
    std::thread t2(process);
    t1.join();
    t2.join();
    return 0;
```

```
}
```

6. Semaphore (Producer-Consumer Problem)

```
#include <iostream>
#include <thread>
#include <semaphore>

#define N 100

std::counting_semaphore<N> empty(N);
std::counting_semaphore<N> full(0);
std::binary_semaphore mutex(1);

void produce_item() {}
void consume_item() {}
void insert_item() {}
void remove_item() {}

void producer() {
    while (true) {
        produce_item();
        empty.acquire();
        mutex.acquire();
        insert_item();
        mutex.release();
        full.release();
    }
}

void consumer() {
    while (true) {
        full.acquire();
        mutex.acquire();
        remove_item();
        mutex.release();
        empty.release();
        consume_item();
    }
}
```

7. Resource Allocation Graph (Cycle Detection)

```
#include <iostream>
#include <vector>

using namespace std;

bool isCyclicUtil(int v, vector<vector<int>>& adj, vector<bool>& visited, vector<bool>&
    recStack) {
    if (!visited[v]) {
        visited[v] = true;
        recStack[v] = true;

        for (int i : adj[v]) {
            if (!visited[i] && isCyclicUtil(i, adj, visited, recStack))
                return true;
            else if (recStack[i])
                return true;
        }
    }
    recStack[v] = false;
    return false;
}

bool isCyclic(int V, vector<vector<int>>& adj) {
    vector<bool> visited(V, false);
```

```
    vector<bool> recStack(V, false);

    for (int i = 0; i < V; i++)
        if (isCyclicUtil(i, adj, visited, recStack))
            return true;
    return false;
}
```

8. Circular Wait Prevention

```
#include <iostream>
#include <mutex>
#include <thread>

std::mutex r1, r2, r3;

void process_A() {
    std::scoped_lock lock(r1, r2);
    // Critical Section
}

void process_B() {
    std::scoped_lock lock(r2, r3);
    // Critical Section
}

void process_C() {
    std::scoped_lock lock(r1, r3);
    // Critical Section
}
```

9. Banker's Algorithm

```
#include <iostream>
#include <vector>

using namespace std;

int main() {
    int n = 5, m = 3;
    int alloc[5][3] = {{0, 1, 0}, {2, 0, 0}, {3, 0, 2}, {2, 1, 1}, {0, 0, 2}};
    int max[5][3] = {{7, 5, 3}, {3, 2, 2}, {9, 0, 2}, {2, 2, 2}, {4, 3, 3}};
    int avail[3] = {3, 3, 2};

    int f[5], ans[5], ind = 0;
    for (int k = 0; k < n; k++) f[k] = 0;

    int need[5][3];
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++)
            need[i][j] = max[i][j] - alloc[i][j];
    }

    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            if (f[i] == 0) {
                int flag = 0;
                for (int j = 0; j < m; j++) {
                    if (need[i][j] > avail[j]) {
                        flag = 1;
                        break;
                    }
                }
                if (flag == 0) {
                    ans[ind++] = i;
                    for (int y = 0; y < m; y++) avail[y] += alloc[i][y];
                    f[i] = 1;
```

```
                }
            }
        }
    }
    return 0;
}
```

10. Single Instance Deadlock Detection (Wait-for Graph)

```
#include <iostream>
#include <vector>

using namespace std;

bool detectCycle(int v, vector<vector<int>>& graph, vector<int>& state) {
    state[v] = 1;
    for (int u : graph[v]) {
        if (state[u] == 1) {
            return true;
        }
        if (state[u] == 0 && detectCycle(u, graph, state)) {
            return true;
        }
    }
    state[v] = 2;
    return false;
}

bool hasDeadlock(int V, vector<vector<int>>& graph) {
    vector<int> state(V, 0);
    for (int i = 0; i < V; ++i) {
        if (state[i] == 0) {
            if (detectCycle(i, graph, state)) {
                return true;
            }
        }
    }
    return false;
}
```

11. Several Instance Deadlock Detection

```
#include <iostream>
#include <vector>

using namespace std;

int main() {
    int n = 5, m = 3;
    int alloc[5][3] = {{0, 1, 0}, {2, 0, 0}, {3, 0, 3}, {2, 1, 1}, {0, 0, 2}};
    int req[5][3] = {{0, 0, 0}, {2, 0, 2}, {0, 0, 0}, {1, 0, 0}, {0, 0, 2}};
    int avail[3] = {0, 0, 0};

    vector<bool> finish(n, false);

    for (int i = 0; i < n; i++) {
        bool allZero = true;
        for (int j = 0; j < m; j++) {
            if (alloc[i][j] != 0) allZero = false;
        }
        if (allZero) finish[i] = true;
    }

    int count = 0;
    while (count < n) {
        bool found = false;
        for (int i = 0; i < n; i++) {
```

```
            if (!finish[i]) {
                bool canSatisfy = true;
                for (int j = 0; j < m; j++) {
                    if (req[i][j] > avail[j]) {
                        canSatisfy = false;
                        break;
                    }
                }
                if (canSatisfy) {
                    for (int j = 0; j < m; j++) avail[j] += alloc[i][j];
                    finish[i] = true;
                    found = true;
                    count++;
                }
            }
        }
        if (!found) break;
    }

    bool deadlock = false;
    for (int i = 0; i < n; i++) {
        if (!finish[i]) deadlock = true;
    }

    return 0;
}
```